



Lecture 3-4: VI XỬ LÝ TRÊN SOC FPGA

Nội dung

2.1. GIỚI THIỆU

2.1.1. Vi xử lý đa năng lõi mềm

2.1.2. Vi xử lý đa năng lõi cứng

2.1.3. Vi xử lý chuyên dụng

2.1.4. So sánh các loại vi xử lý

2.2. CÁC KIẾN TRÚC VI XỬ LÝ LỖI MỀM TRÊN FPGA

2.2.1. Kiến trúc vi xử lý Nios II

2.2.2. Kiến trúc vi xử lý MicroBlaze

2.2.3. Cách kết nối vi xử lý mềm vào trong hệ thống SoC.

2.3. KIẾN TRÚC VI XỬ LÝ ĐA NĂNG LỖI CỨNG TRÊN FPGA

2.3.1. CPU lõi cứng trên Altera FPGA

2.3.2. CPU lõi cứng trên Xilinx FPGA

2.3.3. Cách kết nối vi xử lý cứng vào trong hệ thống SoC

2.4. VI XỬ LÝ CHUYÊN DỤNG CHO HỌC SÂU TRÊN SOC FPGA

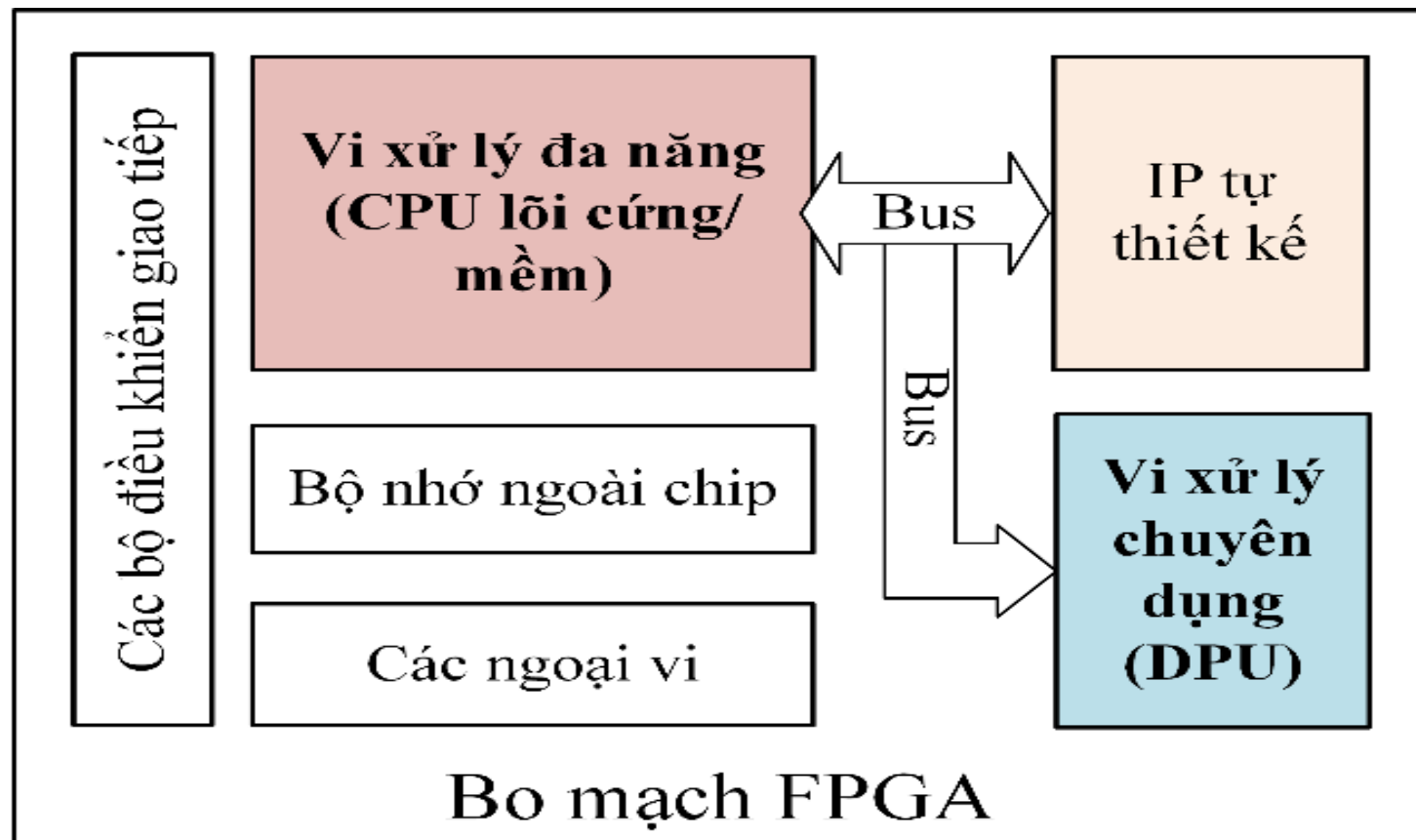
2.4.1. Kiến trúc của DPU

2.5. VI XỬ LÝ TỰ THIẾT KẾ

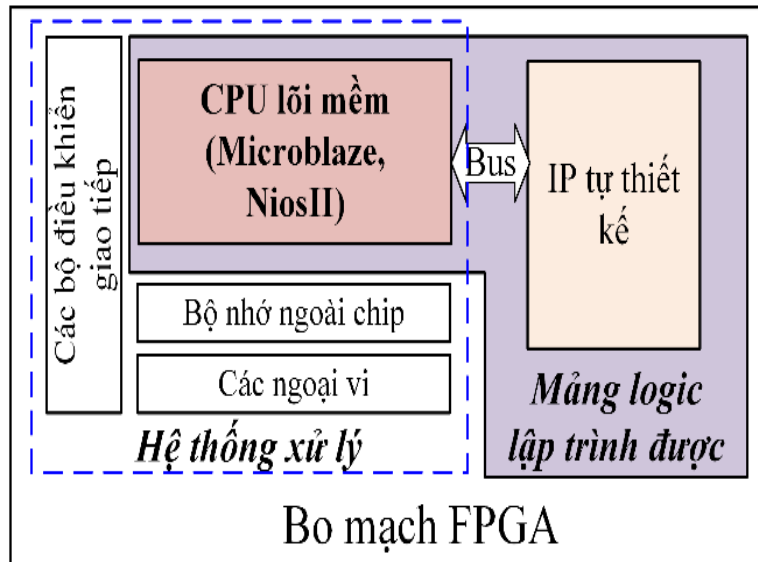
2.6. TÓM TẮT

2.7. CÂU HỎI VÀ BÀI TẬP

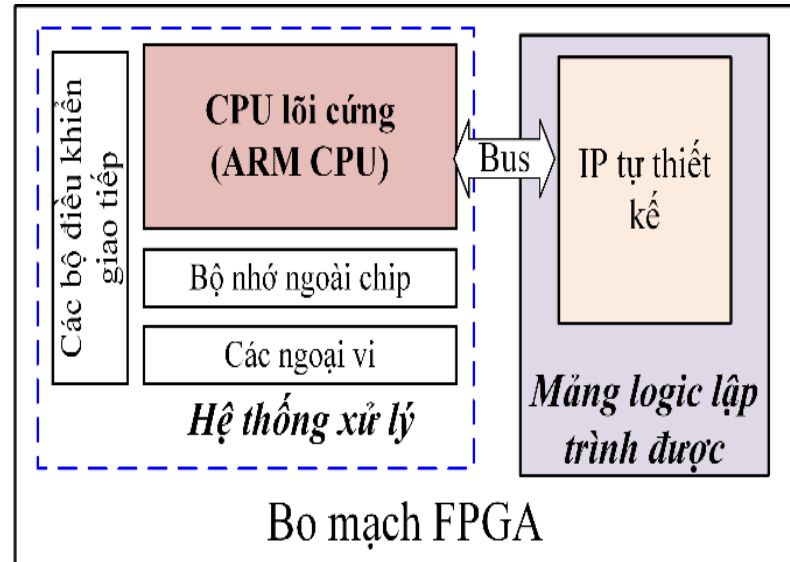
Giới thiệu



Giới thiệu



(a)



(b)

Giới thiệu

Vi xử lý lõi mềm: MicroBlaze của Xilinx hoặc Nios II của Altera (Intel)

■ **Ưu điểm:**

- ✓ Linh hoạt và có thể tùy chỉnh: CPU lõi mềm cho phép thay đổi cấu trúc và các tính năng của vi xử lý để đáp ứng yêu cầu cụ thể của từng ứng dụng.
- ✓ Tiết kiệm chi phí và diện tích
- ✓ Khả năng mở rộng và thích ứng: CPU lõi mềm có thể dễ dàng mở rộng và điều chỉnh khi hệ thống phát triển hoặc có sự thay đổi trong yêu cầu ứng dụng.
- ✓ Tích hợp dễ dàng với các phần cứng khác

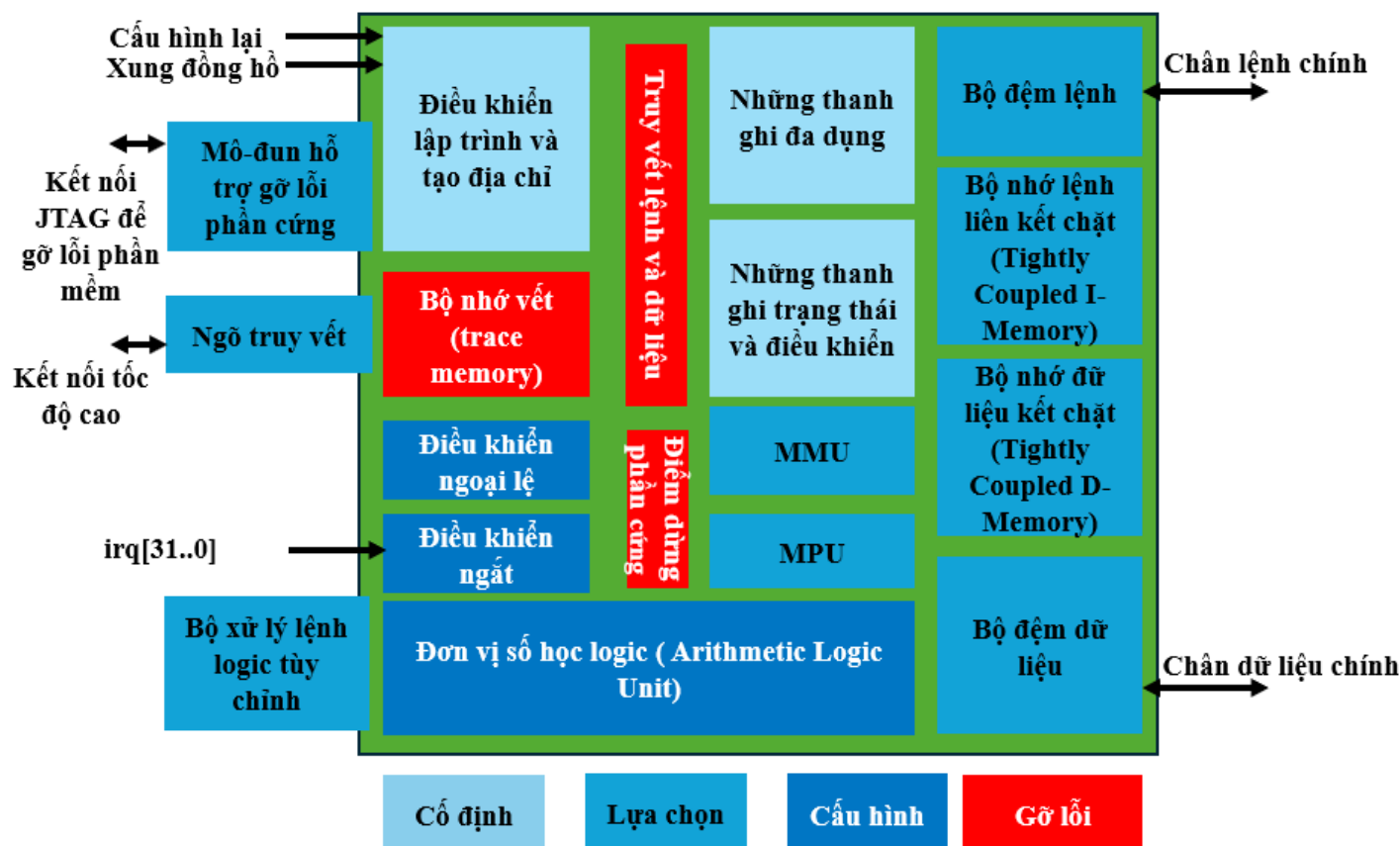
Giới thiệu

✓ **Nhược điểm:**

- ❖ Tần số hoạt động thấp: CPU lõi mềm thường có tần số hoạt động trong khoảng 100-200 MHz
- ❖ Hiệu suất thấp hơn CPU lõi cứng
- ❖ Tiêu thụ tài nguyên FPGA: CPU lõi mềm sử dụng tài nguyên FPGA như logic cells, bộ nhớ, và các I/O blocks.
- ❖ Tiêu thụ năng lượng cao hơn: CPU lõi mềm, mặc dù linh hoạt, có thể tiêu thụ năng lượng cao hơn so với CPU lõi cứng
- ❖ Khả năng xử lý song song hạn chế: CPU lõi mềm yếu hơn nhiều so với CPU lõi cứng về mặt hiệu suất tính toán.

Các kiến trúc vi xử lý lõi mềm trên FPGA

- **Nios II**: tuân theo mô hình RISC pipelined cổ điển, bao gồm 32 thanh ghi đa dụng, ba định dạng lệnh (R, I, J) và hỗ trợ lệnh 32-bit cùng với bus dữ liệu 32-bit



Các kiến trúc vi xử lý lỗi mềm trên FPGA

■ Nios II

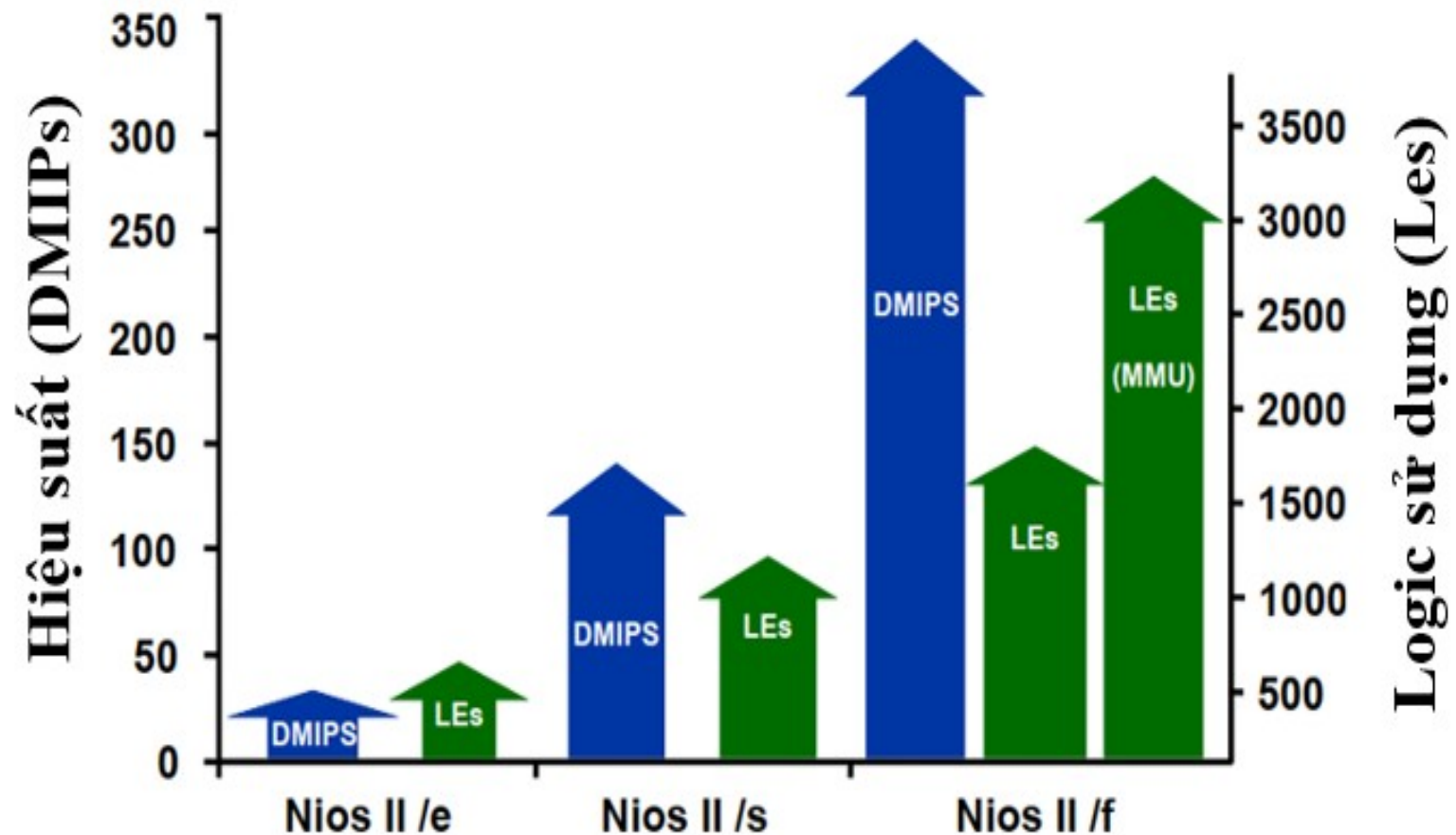
Bảng 2.3: So sánh các loại CPU Nios II

	Nios II /f (Nhanh)	Nios II /s (Tiêu chuẩn)	Nios II /e (Tiết kiệm)
Thông lượng lệnh (Tối đa)	1 lệnh / chu kỳ	1 lệnh / chu kỳ	1 lệnh / 6 chu kỳ
Số lệnh hoạt động tối đa trong pipeline	6	5	1
Bộ nhân & dịch bit bằng phần cứng	1 chu kỳ	3 chu kỳ	Mô phỏng bằng phần mềm
Dự đoán nhánh	Động	Tĩnh	Không có
Bộ nhớ đệm lệnh	Cấu hình được	Cấu hình được	Không có
Bộ nhớ đệm dữ liệu	Cấu hình được	Không có	Không có
Yêu cầu tài nguyên logic (Số LEs điển hình)	1800 không hoặc không có MMU hoặc 3200 có MMU	1200	600
Lệnh tùy chỉnh	Tối đa 256	Tối đa 256	Không có



Các kiến trúc vi xử lý lõi mềm trên FPGA

■ Nios II



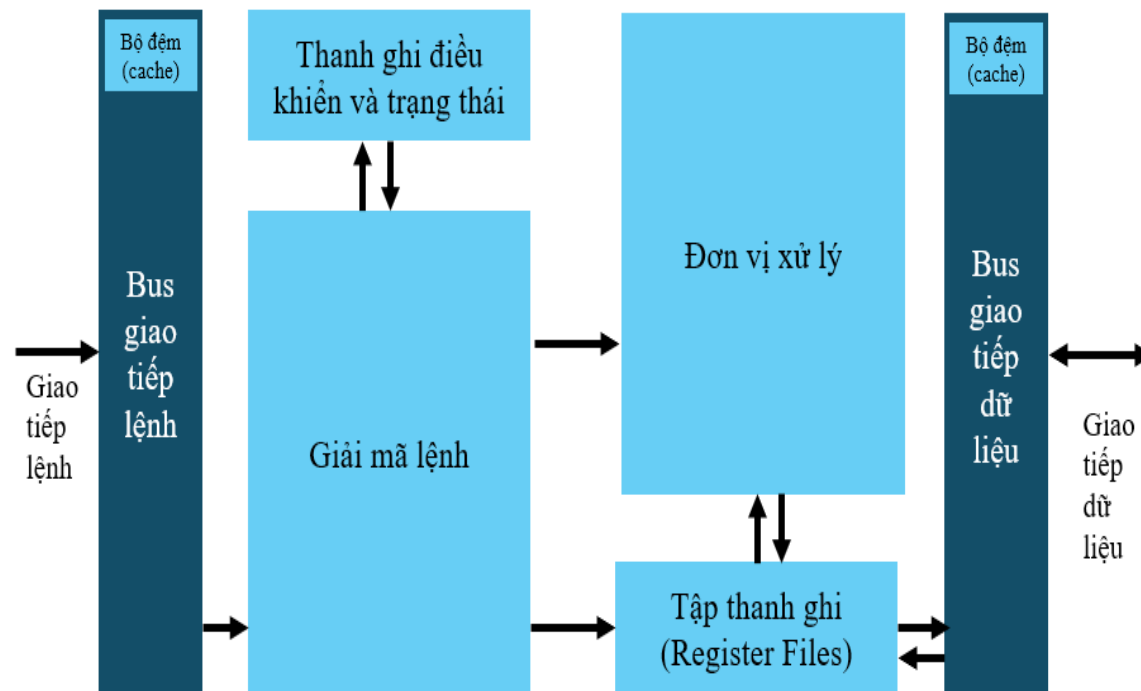
Các kiến trúc vi xử lý lỗi mềm trên FPGA

MicroBlaze

- MicroBlaze là một bộ vi xử lý mềm (soft CPU) do AMD (trước đây là Xilinx) phát triển
- họ như Spartan-6, Artix-7, Virtex-7.
- Đây là một vi xử lý dựa trên kiến trúc Harvard RISC, hỗ trợ cả cấu hình 32-bit và 64-bit,
- MicroBlaze sở hữu tập lệnh độc lập (orthogonal), cho phép các lệnh sử dụng bất kỳ thanh ghi nào một cách linh hoạt, tối ưu hóa việc lập trình và hiệu suất.

Các kiến trúc vi xử lý lỗi mềm trên FPGA

MicroBlaze



Các kiến trúc vi xử lý lỗi mềm trên FPGA

MicroBlaze sử dụng hai định dạng lệnh chính: Loại A (Type A) và Loại B (Type B):

- Loại A: Được sử dụng chủ yếu cho các lệnh thao tác giữa các thanh ghi (register-register). Định dạng này bao gồm mã lệnh (opcode và func), một thanh ghi đích (Rd), và hai thanh ghi nguồn (Ra, Rb).
- Loại B: Được sử dụng cho các lệnh thao tác giữa thanh ghi và giá trị tức thời (immediate value). Định dạng này bao gồm mã lệnh (opcode), một thanh ghi đích (Rd), một thanh ghi nguồn (Ra), và một giá trị tức thời 16-bit làm nguồn dữ liệu.

Loại A	Mã lệnh (opcode)	Rd	Ra	Rb	Chức năng (func)
Loại B	Mã lệnh (opcode)	Rd	Ra	Giá trị tức thời (Immediate value)	

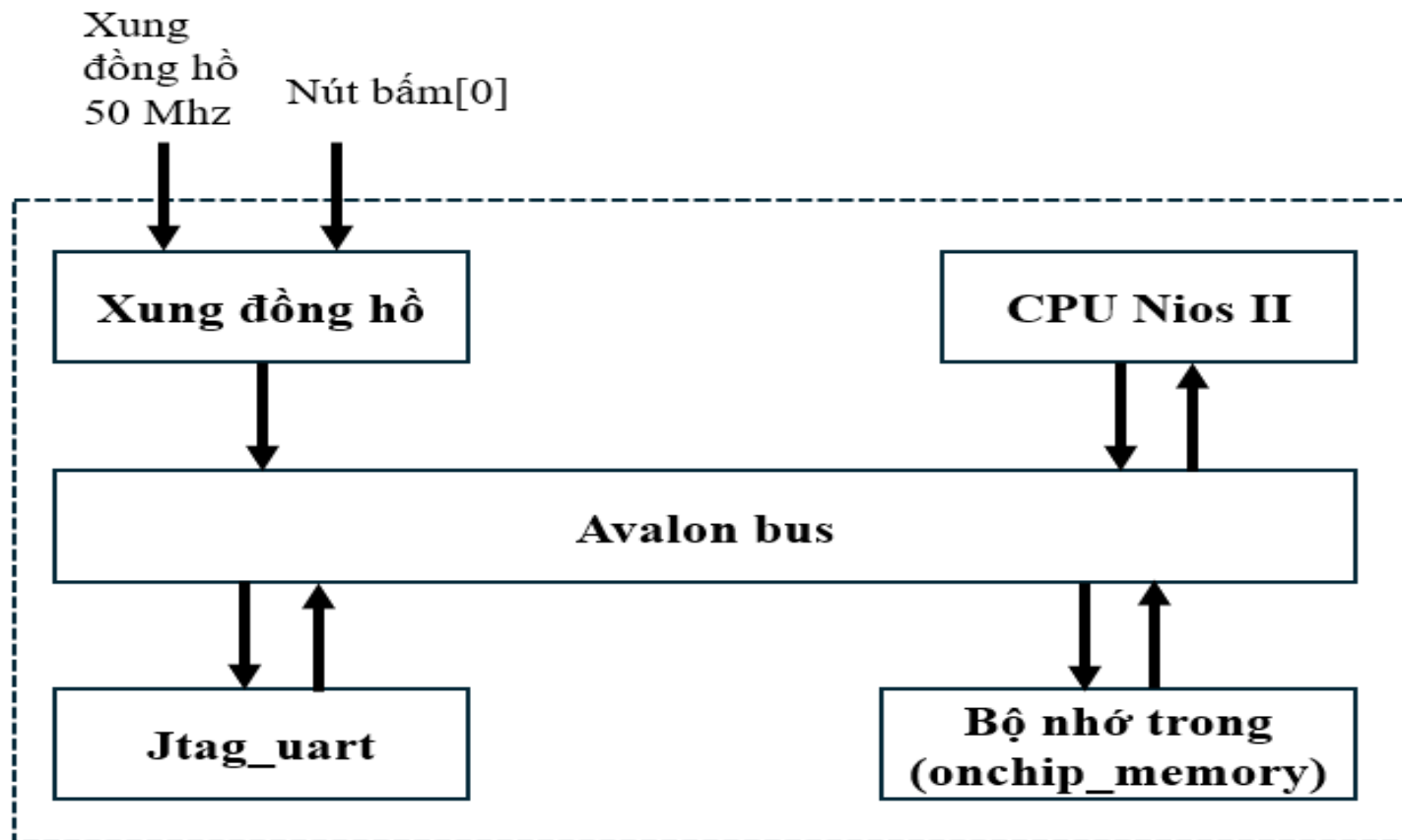


Các kiến trúc vi xử lý lỗi mềm trên FPGA

MicroBlaze

- ✓ Bộ vi xử lý MicroBlaze sử dụng kiến trúc pipeline gồm 3 giai đoạn: nạp lệnh (fetch), giải mã (decode), và hoàn thành (complete).
- ✓ Việc chuyển tiếp dữ liệu (data forwarding), xử lý rẽ nhánh (branches), và dừng pipeline (pipeline stall) đều được phần cứng tự động xác định, giúp tối ưu hóa hiệu suất hoạt động.
- ✓ MicroBlaze hỗ trợ ba kích thước dữ liệu trong bộ nhớ: 8-bit (Byte), 16-bit (Halfword), và 32-bit (Word).
- ✓ Đây là bộ xử lý Big-Endian
- ✓ MicroBlaze hỗ trợ bề rộng bus 32-bit và là một bộ xử lý RISC có tập lệnh riêng biệt cho truy cập bộ nhớ và dữ liệu.

Cách kết nối vi xử lý mềm vào trong hệ thống SoC



Cách kết nối vi xử lý mềm vào trong hệ thống SoC

Qsys - system.qsys (D:\lab\bai\system.qsys)

File Edit System View Tools Help

Component Library

Project

- New Component...
- System

Library

- Config-Bypass App Example
- Bridges
- Bridges and Adapters
- Clock and Reset
- Configuration & Programming
- DSP
- Embedded Processors
- Interface Protocols
- Memories and Memory Controller
- Merlin Components
- Microcontroller Peripherals
- Peripherals
- PLL
- Qsys Interconnect
- Verification
- Window Bridge

New... Edit... Add...

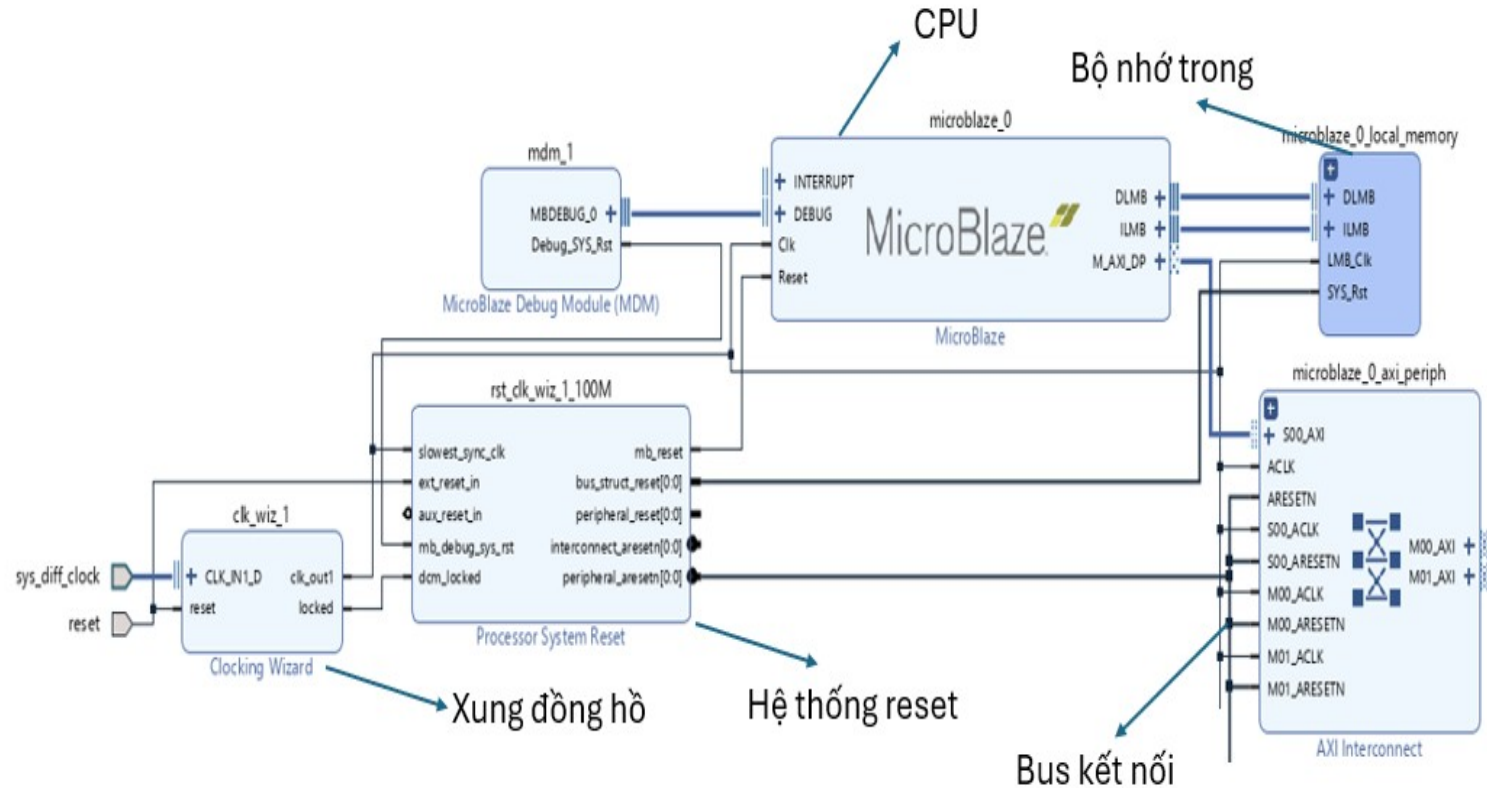
System Contents Address Map Clock Settings Project Settings Instance Parameters System Inspector HDL Example Generation

Use	Connections	Name	Description	Export	Clock	Base
☒		clock	Clock Source			
		clk_in	Clock Input	clk		
		clk_in_reset	Reset Input	reset		
		clk	Clock Output	Double-click to export	clock	
		clk_reset	Reset Output	Double-click to export		
☒		nios2_qsys_0	Nios II Processor			
		clk	Clock Input	Double-click to export	clock	
		reset_n	Reset Input	Double-click to export	[clk]	
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]	
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]	
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]	
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	
		custom_instruction_m...	Custom Instruction Master	Double-click to export		
☒		onchip_memory2_0	On-Chip Memory (RAM or ROM)			
		clk1	Clock Input	Double-click to export	clock	
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	0x0000_8000
		reset1	Reset Input	Double-click to export	[clk1]	
☒		jtag_uart_0	JTAG UART			
		clk	Clock Input	Double-click to export	clock	
		reset	Reset Input	Double-click to export	[clk]	
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0001_1008

```
#include <stdio.h>

void main(){
    printf("Bai 1 thuc hanh SoC\n");
}
```

Cách kết nối vi xử lý mềm vào trong hệ thống SoC



```
#include <stdio.h>

void main() {
    printf("Bai 1 thuc hanh SoC\n");
}
```




1. Nêu cấu tạo chính của CPU lõi cứng
2. Kể tên một số dòng CPU có tích hợp trên board FPGA hiện nay

Giới thiệu

Vi xử lý lõi cứng: ARM Cortex-A9, ARM Cortex-A53

■ **Ưu điểm:**

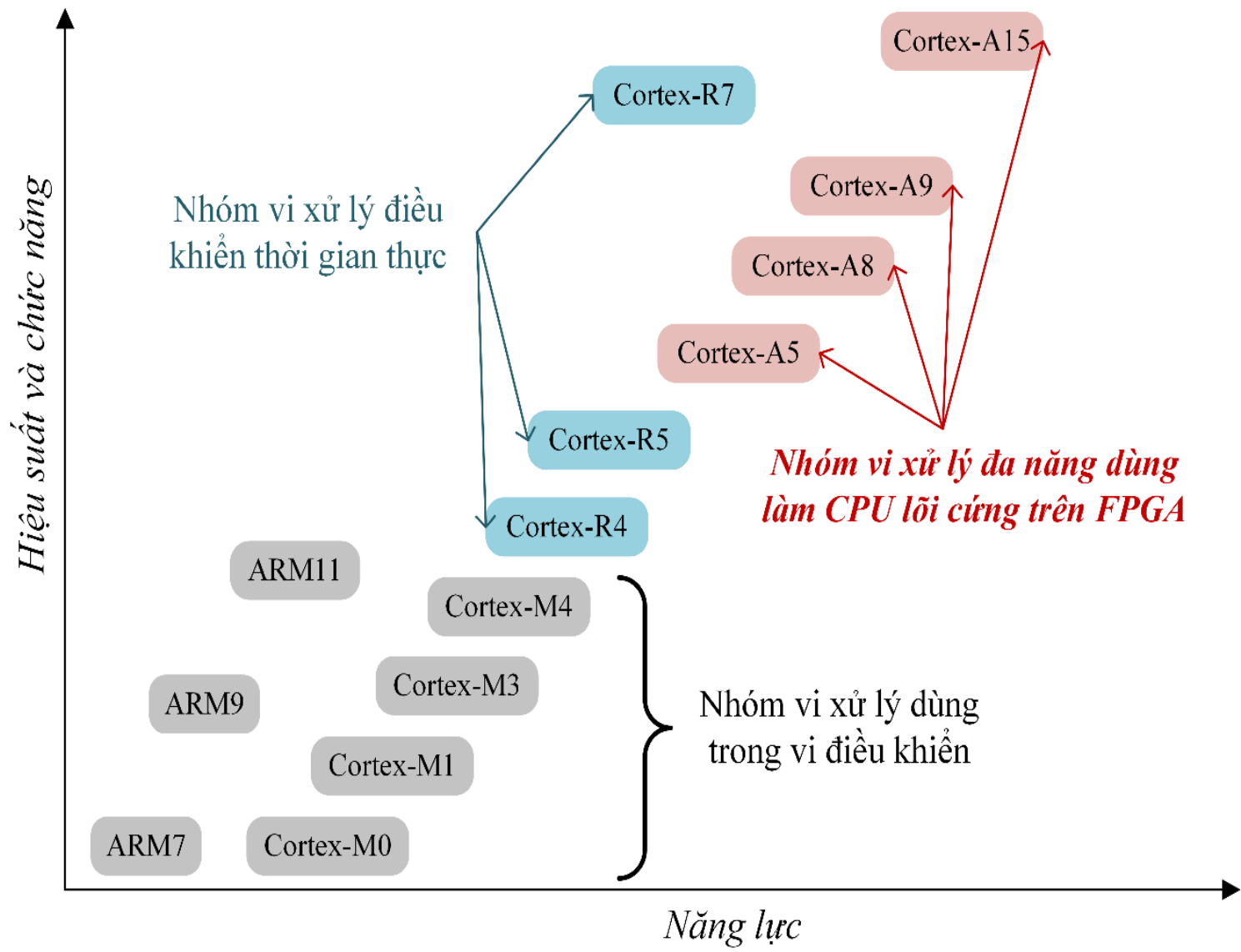
- ✓ Hiệu suất cao và khả năng tối ưu hóa phần cứng
- ✓ Tiết kiệm chi phí và diện tích: Việc tích hợp CPU lõi cứng trực tiếp vào SoC FPGA giúp giảm thiểu diện tích và chi phí
- ✓ Khả năng chạy hệ điều hành phức tạp: CPU lõi cứng có khả năng hỗ trợ các hệ điều hành phức tạp như Linux, mở ra khả năng phát triển ứng dụng đa dạng.
- ✓ Tính tương thích cao với các công nghệ khác: CPU lõi cứng dễ dàng tích hợp với nhiều công nghệ và phần cứng khác nhau
- ✓ Tiết kiệm năng lượng: Các CPU lõi cứng trên SoC FPGA được tối ưu hóa để tiết kiệm năng lượng.

Giới thiệu

Vi xử lý lõi cứng: ARM Cortex-A9, ARM Cortex-A53

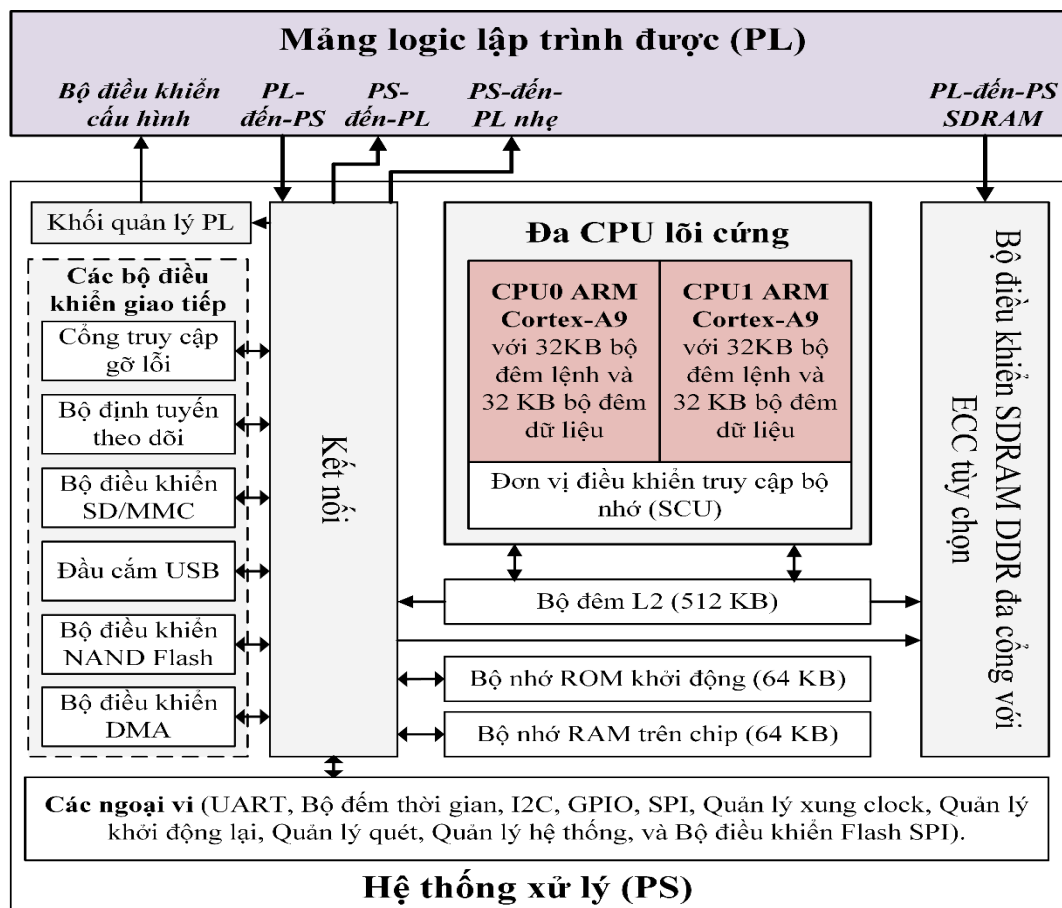
■ **Nhược điểm:**

- ❖ Khả năng tùy chỉnh phần cứng hạn chế: CPU lõi cứng không thể thay đổi cấu trúc phần cứng như các giải pháp tùy chỉnh hoàn toàn.
- ❖ Khả năng xử lý song song hạn chế: Mặc dù CPU lõi cứng trên FPGA vẫn đủ mạnh để tính toán và xử lý nhanh trong hệ thống nhúng, nhưng chúng không tối ưu cho các tác vụ tính toán song song quy mô lớn
- ❖ Chi phí phát triển và phần mềm: Việc tích hợp CPU lõi cứng vào hệ thống SoC FPGA có thể yêu cầu các công cụ phát triển phần mềm và các giấy phép phần mềm, điều này có thể làm tăng chi phí tổng thể của dự án.



Kiến trúc vi xử lý đa năng lõi cứng trên FPGA

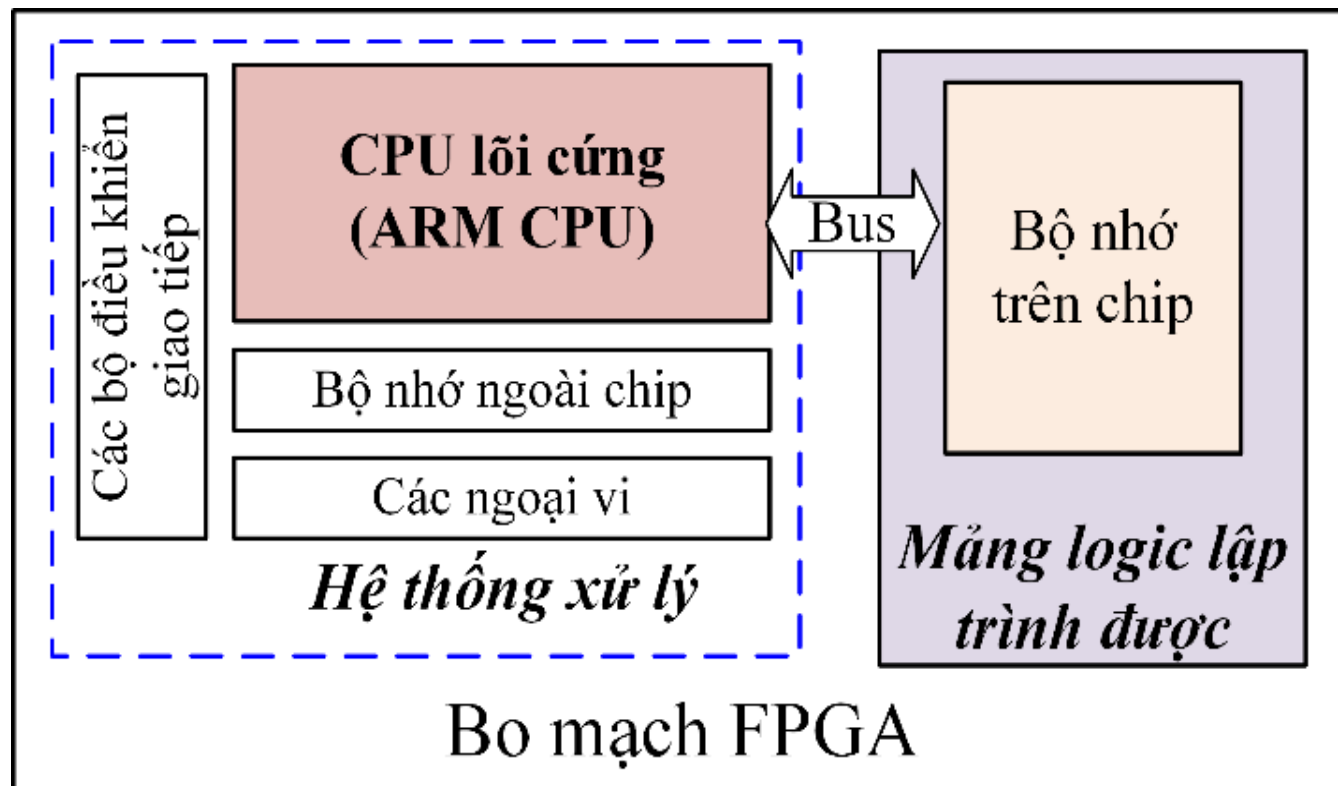
Các dòng Cyclone V, Arria 10 SoC, và Stratix 10 SoC là những dòng FPGA Altera nổi tiếng sử dụng CPU lõi cứng để xử lý.



Kiến trúc vi xử lý đa năng lõi cứng trên FPGA

- Đặc điểm của CPU lõi cứng trên Xilinx FPGA tương tự như Đa CPU lõi cứng của Altera FPGA vì cả hai đều sử dụng cùng kiến trúc ARM Cortex-A9.
- ARM Cortex-A9 là một vi xử lý đa lõi 32-bit, hỗ trợ tần số tối đa lên đến 800 MHz
- Cấu trúc này bao gồm hai lõi ARM Cortex-A9, mỗi lõi có bộ đệm lệnh 32KB và bộ đệm dữ liệu 32KB,
- Vi xử lý ARM Cortex-A9 hỗ trợ các phép toán đơn lệnh đa dữ liệu SIMD với 128-bit
- ARM Cortex-A9 còn có bộ điều khiển bộ nhớ (MMU) giúp quản lý bộ nhớ hiệu quả.
- Cấu trúc bộ nhớ của vi xử lý này cũng bao gồm bộ đệm cấp 2 (L2) 512KB, được chia sẻ giữa các lõi để tăng hiệu suất và giảm tắc nghẽn trong quá trình xử lý

Kiến trúc vi xử lý đa năng lõi cứng trên FPGA



Giới thiệu

Vi xử lý chuyên dụng *như DPU*: được thiết kế đặc biệt để tăng tốc các tác vụ tính toán phức tạp, đặc biệt là trong lĩnh vực trí tuệ nhân tạo (AI) và học máy (ML)

■ Ưu điểm:

- ✓ Tăng tốc AI/ML: DPU được thiết kế để tăng tốc các tác vụ AI và ML, giúp giảm thời gian xử lý và tăng hiệu suất cho các ứng dụng như nhận diện hình ảnh, phân tích dữ liệu và các tác vụ học sâu khác.
- ✓ Tiết kiệm năng lượng: So với việc sử dụng CPU hoặc GPU truyền thống, DPU tiêu thụ ít năng lượng hơn khi xử lý các tác vụ AI/ML, giúp tiết kiệm chi phí vận hành và giảm thiểu tác động đến môi trường.
- ✓ Khả năng mở rộng và linh hoạt: DPU có thể được tích hợp vào các hệ thống SoC, FPGA, cho phép mở rộng và tùy chỉnh theo nhu cầu cụ thể của ứng dụng, mang lại sự linh hoạt cao trong thiết kế hệ thống.

Giới thiệu

Vi xử lý chuyên dụng *như DPU*: được thiết kế đặc biệt để tăng tốc các tác vụ tính toán phức tạp, đặc biệt là trong lĩnh vực trí tuệ nhân tạo (AI) và học máy (ML)

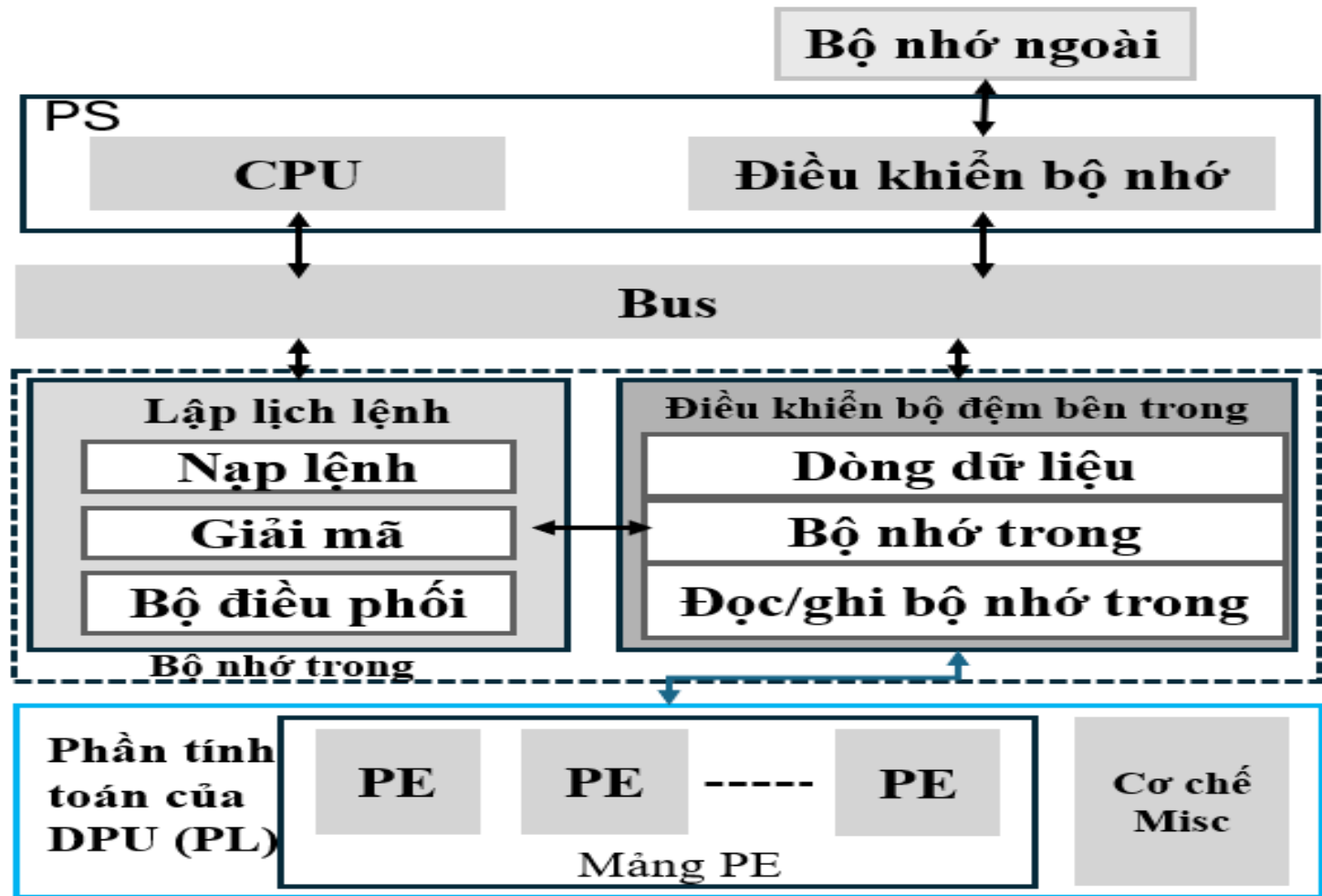
■ **Nhược điểm:**

- ❖ Khả năng tùy chỉnh phần cứng hạn chế: Mặc dù DPU cung cấp khả năng tăng tốc cho các tác vụ AI/ML, nhưng khả năng tùy chỉnh phần cứng để đáp ứng các yêu cầu đặc thù có thể bị hạn chế so với việc sử dụng các giải pháp phần cứng tùy chỉnh hoàn toàn.
- ❖ Chi phí phát triển và tích hợp: Việc tích hợp DPU vào hệ thống có thể đòi hỏi chi phí phát triển và tích hợp cao hơn
- ❖ Tiêu tốn tài nguyên phần cứng: DPU chiếm dụng một phần tài nguyên logic và bộ nhớ của FPGA

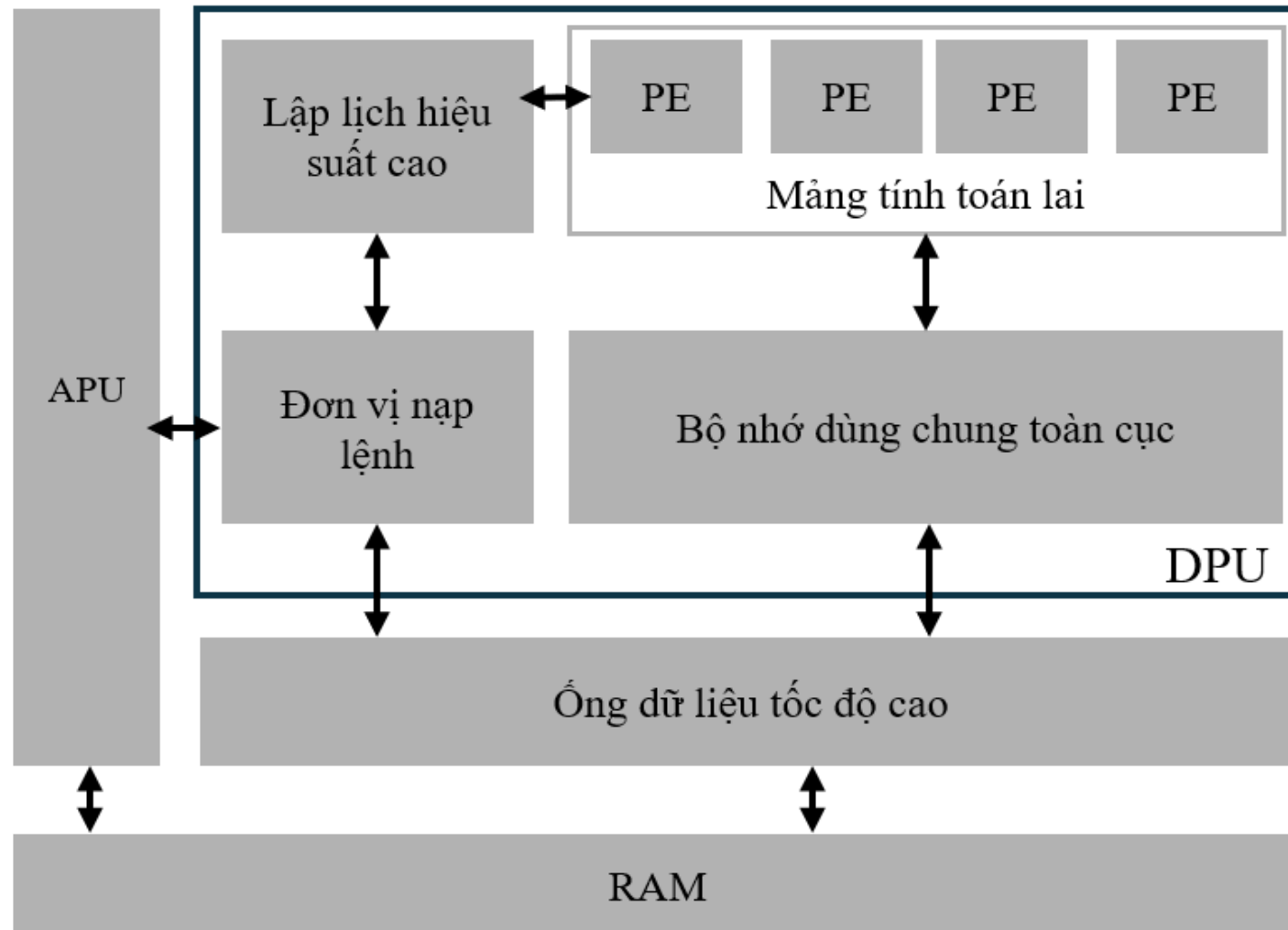
Vi xử lý chuyên dụng cho học sâu trên SoC FPGA

- ✓ Đơn vị vi xử lý học sâu (DPU- Deep Learning Processing Unit) là một IP cứng
- ✓ được thiết kế để tăng tốc các tác vụ AI, đặc biệt là các mô hình học sâu (Deep Learning) trên FPGA.
- ✓ Kiến trúc của DPU được tối ưu hóa cho các phép toán tensor, hỗ trợ các định dạng dữ liệu như INT8
- ✓ DPU thường bao gồm nhiều phần tử xử lý (PE - Processing Elements) hoạt động song song,
- ✓ Trên các nền tảng FPGA của Xilinx, DPU được tích hợp trong các dòng Zynq UltraScale+ MPSoC và Versal ACAP, hỗ trợ thông qua công cụ Vitis AI.
- ✓ Trong khi đó, Intel (Altera) cung cấp các giải pháp tương tự dựa trên OpenVINO và sử dụng các bộ gia tốc AI trong các FPGA như Stratix 10 hoặc Agilex.

Vi xử lý chuyên dụng cho học sâu trên SoC FPGA



Vi xử lý chuyên dụng cho học sâu trên SoC FPGA

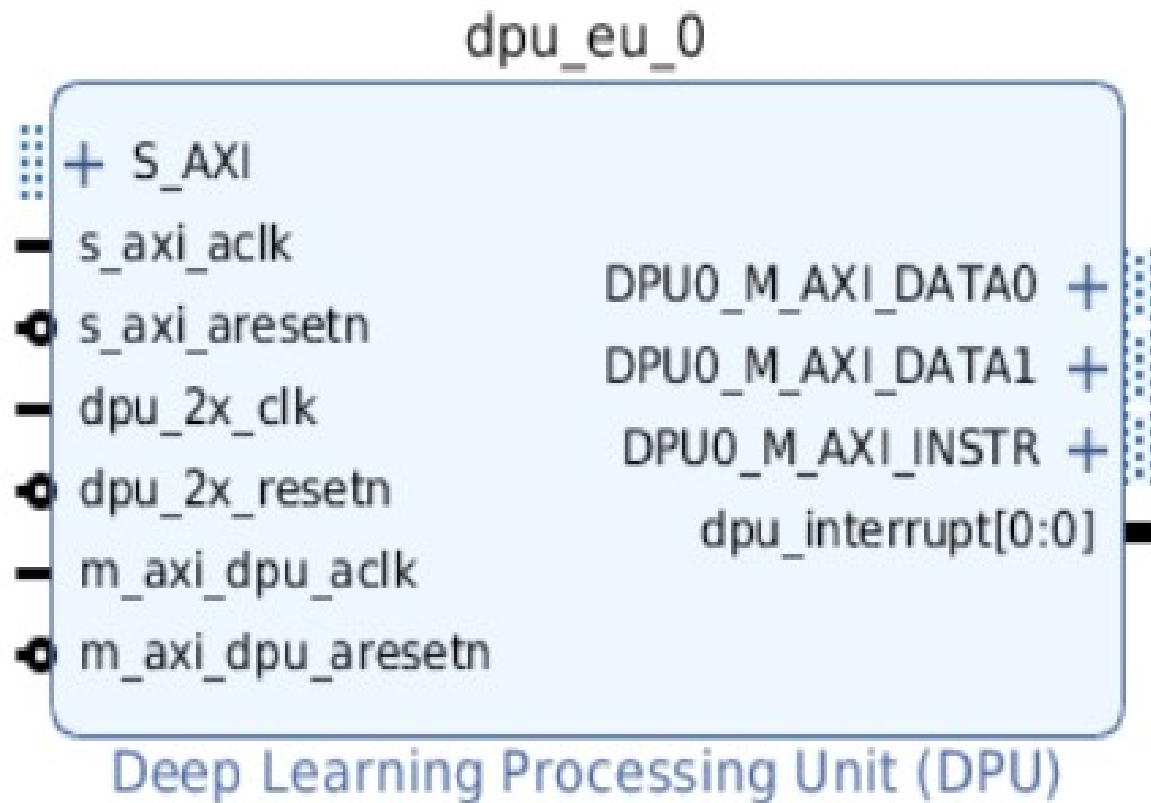


Vì xử lý chuyên dụng cho học sâu trên SoC FPGA

□ Các tính năng nổi bật về chức năng:

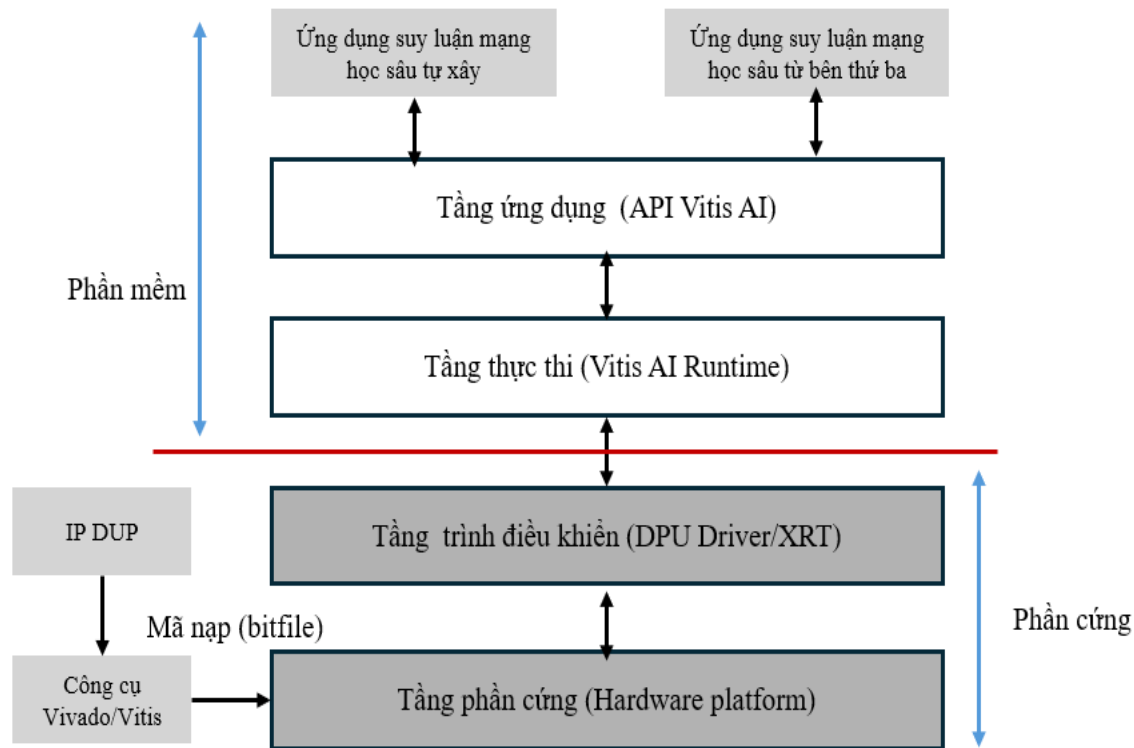
- Kiến trúc phần cứng có thể cấu hình: DPU hỗ trợ nhiều đơn vị tính toán (computational units) hoặc số lượng phép nhân cộng tích lũy (MACs -Multiply-Accumulate Units) khác nhau như B512, B800, B1024, B1152, B1600, B2304, B3136, và B4096.
- Hỗ trợ tối đa bốn lõi đồng nhất (homogeneous cores): Giúp tăng cường khả năng xử lý song song, phù hợp với các bài toán AI phức tạp.
- Các phép toán xử lý chuyên dụng: DPU hỗ trợ các phép toán phổ biến trong mạng học sâu như tích chập (convolution), phép tích chập ngược (deconvolution), và tích chập theo chiều sâu (depthwise convolution).
- Kỹ thuật pooling: Bao gồm cả max pooling và average pooling để giảm kích thước dữ liệu và tăng hiệu quả tính toán.
- Hàm kích hoạt đa dạng: DPU hỗ trợ các hàm kích hoạt phổ biến như ReLU, ReLU6, và Leaky ReLU, phù hợp với nhiều kiến trúc mạng nơ-ron khác nhau.
- Các phép toán cơ bản khác: Bao gồm nối dữ liệu (concat) và cộng từng phần tử (elementwise-sum) để xử lý dữ liệu hiệu quả trong các tầng mạng phức tạp.

Vi xử lý chuyên dụng cho học sâu trên SoC FPGA



Vi xử lý chuyên dụng cho học sâu trên SoC FPGA

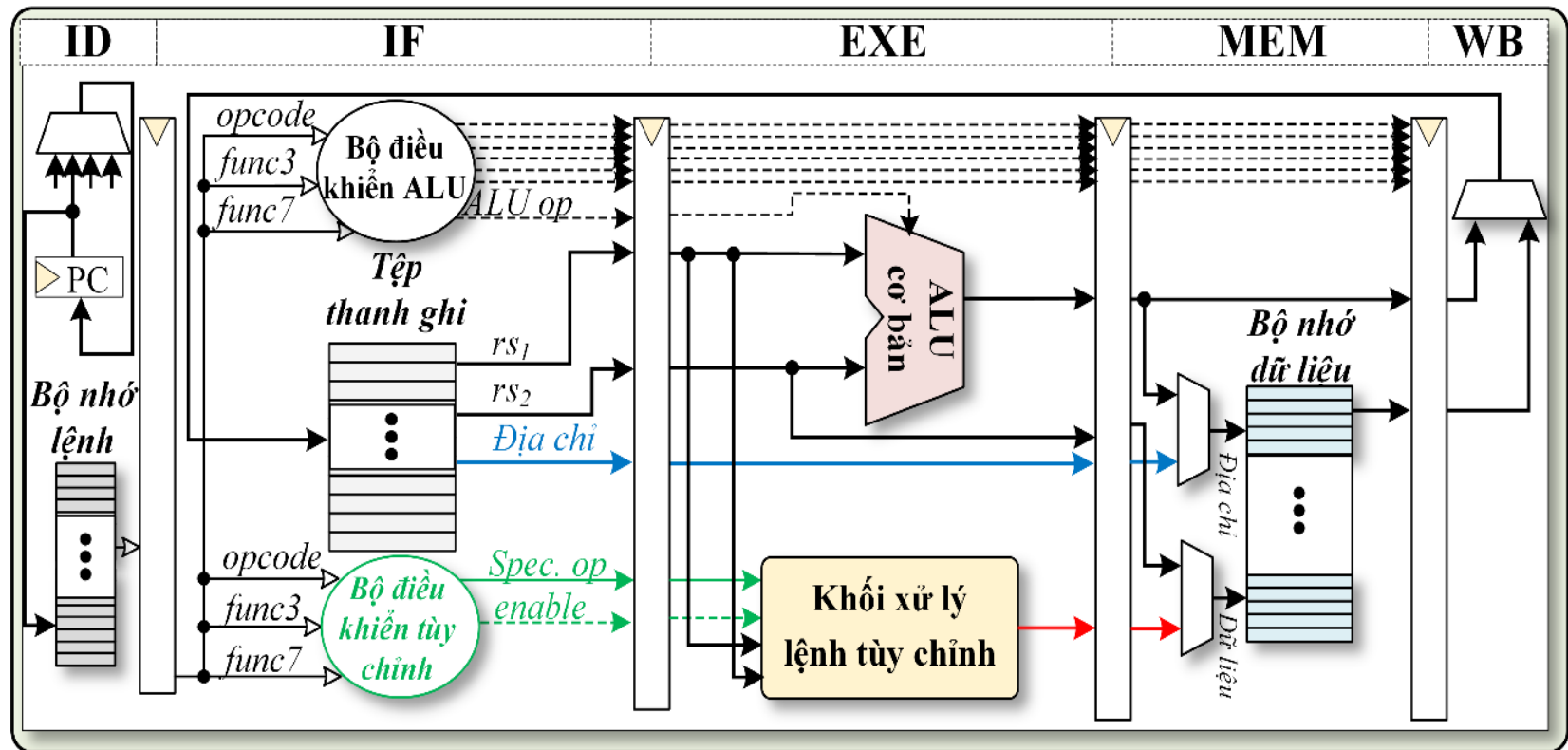
Quy trình hiện thực ứng dụng AI trên DPU



Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

- RISC-V được đề xuất lần đầu tiên bởi nhóm nghiên cứu tại Đại học California, Berkeley vào năm 2010
- RISC-V là một mã nguồn mở
- Điểm mạnh của RISC-V so với ARM là tính linh hoạt và khả năng mở rộng dễ dàng, giúp giảm thiểu chi phí bản quyền vì không bị ràng buộc bởi các giấy phép độc quyền như ARM.
- Cộng đồng mã nguồn mở RISC-V liên tục phát triển và cung cấp nhiều công cụ và tài nguyên hỗ trợ, tạo ra cơ hội lớn cho việc áp dụng RISC-V trong các ứng dụng từ đơn giản đến phức tạp.

Tích hợp vi xử lý tự thiết kế vào hệ thống SoC



Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

- RISC-V có một hệ thống bộ lệnh (ISA - Instruction Set Architecture). Các loại bộ lệnh chính của RISC-V bao gồm các tập lệnh cơ bản như :
 - I (Integer - Số nguyên),
 - E (Embedded - Nhúng),
 - V (Vector - Vecto),
 - C (Compressed - Nén),
 - M (Multiplication and Division - Nhân và Chia),
 - A (Atomic - Tập lệnh Hạt nhân),
 - F (Single-Precision Floating-Point - Dấu phẩy động độ chính xác đơn),
 - D (Double-Precision Floating-Point - Dấu phẩy động độ chính xác kép).
 - Z (Standard Extensions - Các Mở rộng Tiêu chuẩn).



Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

Tập lệnh I

				imme[11:0]		rs1	funct3	rd	opcode							
				31	20	19	15	14	12	11	7	6	0			
I-Type	FLW	FLW frd, offset(rs1)	F	imme[11:0]		rs1		010		frd		0000111		LOAD_FP		
	LB	LB rd, offset(rs1)	I	imme[11:0]		rs1		000		rd		0000011		LOAD		
	LH	LH rd, offset(rs1)	I	imme[11:0]		rs1		001		rd		0000011		LOAD		
	LW	LW rd, offset(rs1)	I	imme[11:0]		rs1		010		rd		0000011		LOAD		
	LBU	LBU rd, offset(rs1)	I	imme[11:0]		rs1		100		rd		0000011		LOAD		
	LHU	LHU rd, offset(rs1)	I	imme[11:0]		rs1		101		rd		0000011		LOAD		
	ADDI	ADDI rd, rs1, imme	I	imme[11:0]		rs1		000		rd		0010011		OP_IMM		
	SLTI	SLTI rd, rs1, imme	I	imme[11:0]		rs1		010		rd		0010011		OP_IMM		
	SLTIU	SLTIU rd, rs1, imme	I	imme[11:0]		rs1		011		rd		0010011		OP_IMM		
	XORI	XORI rd, rs1, imme	I	imme[11:0]		rs1		100		rd		0010011		OP_IMM		
	ORI	ORI rd, rs1, imme	I	imme[11:0]		rs1		110		rd		0010011		OP_IMM		
	ANDI	ANDI rd, rs1, imme	I	imme[11:0]		rs1		111		rd		0010011		OP_IMM		
	JALR	jalr rd, offset(rs1)	I	imme[11:0]		rs1		000		rd		1100111		JALR		
	NOP	NOP	I	000000000000		00000		000		00000		0010011		OP_IMM		
				csr		rs1		funct3		rd		opcode				
	CSRRW	CSRRW rd, csr, rs1	Zicr	csr		rs1		001		rd		1110011		SYSTEM		
	CSRRS	CSRRS rd, csr, rs1	Zicr	csr		rs1		010		rd		1110011		SYSTEM		
	CSRRC	CSRRC rd, csr, rs1	Zicr	csr		rs1		011		rd		1110011		SYSTEM		
				csr		imme[4:0]		funct3		rd		opcode				
	CSRRWI	CSRRWI rd, csr, uimm	Zicr	csr		uimm		101		rd		1110011		SYSTEM		
	CSRRSI	CSRRSI rd, csr, uimm	Zicr	csr		uimm		110		rd		1110011		SYSTEM		
	CSRRCI	CSRRCI rd, csr, uimm	Zicr	csr		uimm		111		rd		1110011		SYSTEM		
				funct7		shamt		rs1		funct3		rd		opcode		
	SLLI	SLLI rd, rs1, imme	I	00000000		shamt		rs1		001		rd		0010011		OP_IMM
	SRLI	SRLI rd, rs1, imme	I	00000000		shamt		rs1		101		rd		0010011		OP_IMM
	SRAI	SRAI rd, rs1, imme	I	01000000		shamt		rs1		101		rd		0010011		OP_IMM
S-Type				imme[11:5]		rs2		rs1		funct3		imme[4:0]		opcode		
	FSW	FSW frs2, offset(rs1)	F	imme[11:5]		frs2		rs1		010		imme[4:0]		0100111		STORE_FP
	SB	SB rs2, offset(rs1)	I	imme[11:5]		rs2		rs1		000		imme[4:0]		0100011		STORE
	SH	SH rs2, offset(rs1)	I	imme[11:5]		rs2		rs1		001		imme[4:0]		0100011		STORE
	SW	SW rs2, offset(rs1)	I	imme[11:5]		rs2		rs1		010		imme[4:0]		0100011		STORE

Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

Tập lệnh I & M

				imme[12 10:5]	rs2	rs1	funct3	imme[4:1 11]	opcode	
B-Type	BEQ	BEQ rs1, rs2, imme	I	imme[12 10:5]	rs2	rs1	000	imme[4:1 11]	1100011	BRANCH
	BNE	BNE rs1, rs2, imme	I	imme[12 10:5]	rs2	rs1	001	imme[4:1 11]	1100011	BRANCH
	BLT	BLT rs1, rs2, imme	I	imme[12 10:5]	rs2	rs1	100	imme[4:1 11]	1100011	BRANCH
	BGE	BGE rs1, rs2, imme	I	imme[12 10:5]	rs2	rs1	101	imme[4:1 11]	1100011	BRANCH
	BLTU	BLTU rs1, rs2, imme	I	imme[12 10:5]	rs2	rs1	110	imme[4:1 11]	1100011	BRANCH
	BGEU	BGEU rs1, rs2, imme	I	imme[12 10:5]	rs2	rs1	111	imme[4:1 11]	1100011	BRANCH
U-Type				imme[31:12]				rd	opcode	
	AUIPC	AUIPC rd, imme	I	imme[31:12]				rd	0010111	AUIPC
	LUI	LUI rd, imme	I	imme[31:12]				rd	0110111	LUI
J-Type				imme[20 10:1 11 19:12]				rd	opcode	
	JAL	JAL rd, offset	I	imme[20 10:1 11 19:12]				rd	1101111	JAL
R-Type				funct7	rs2	rs1	funct3	rd	opcode	
	ADD	ADD rd, rs1, rs2	I	0000000	rs2	rs1	000	rd	0110011	OP
	SUB	SUB rd, rs1, rs2	I	0100000	rs2	rs1	000	rd	0110011	OP
	SLL	SLL rd, rs1, rs2	I	0000000	rs2	rs1	001	rd	0110011	OP
	SLT	SLT rd, rs1, rs2	I	0000000	rs2	rs1	010	rd	0110011	OP
	SLTU	SLTU rd, rs1, rs2	I	0000000	rs2	rs1	011	rd	0110011	OP
	XOR	XOR rd, rs1, rs2	I	0000000	rs2	rs1	100	rd	0110011	OP
	SRL	SRL rd, rs1, rs2	I	0000000	rs2	rs1	101	rd	0110011	OP
	SRA	SRA rd, rs1, rs2	I	0100000	rs2	rs1	101	rd	0110011	OP
	OR	OR rd, rs1, rs2	I	0000000	rs2	rs1	110	rd	0110011	OP
	AND	AND rd, rs1, rs2	I	0000000	rs2	rs1	111	rd	0110011	OP
	MUL	MUL rd, rs1, rs2	M	0000001	rs2	rs1	000	rd	0110011	OP
	MULH	MULH rd, rs1, rs2	M	0000001	rs2	rs1	001	rd	0110011	OP
	MULHSU	MULHSU rd, rs1, rs2	M	0000001	rs2	rs1	010	rd	0110011	OP
	MULHU	MULHU rd, rs1, rs2	M	0000001	rs2	rs1	011	rd	0110011	OP
	DIV	DIV rd, rs1, rs2	M	0000001	rs2	rs1	100	rd	0110011	OP
	DIVU	DIVU rd, rs1, rs2	M	0000001	rs2	rs1	101	rd	0110011	OP
	REM	REM rd, rs1, rs2	M	0000001	rs2	rs1	110	rd	0110011	OP
	REMU	REMU rd, rs1, rs2	M	0000001	rs2	rs1	111	rd	0110011	OP

Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

Tập lệnh C

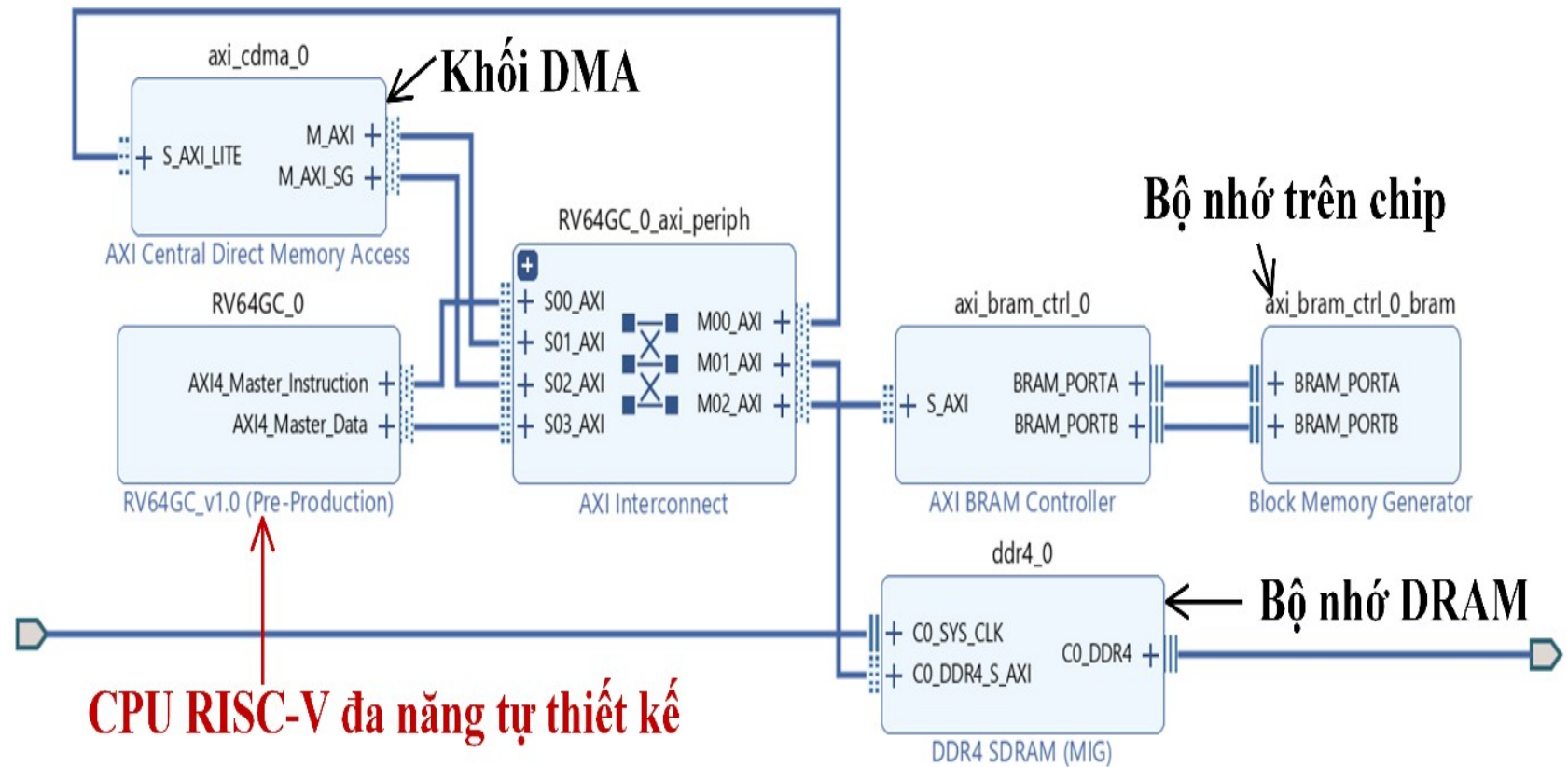
FORMAT	MNEMONIC	SYNTAX	DECOMPRESSED	STANDARD	XLEN 16			
CR-Type					funct4	rd/rs1	rs2	opcode
	C.JR	C.JR rs1	JALR x0, 0(rs1)	C	1000	rs1 # 0	00000	10
	C.JALR	C.JALR rs1	JALR x1, 0(rs1)	C	1001	rs1 # 0	00000	10
	C.MV	C.MV rd, rs2	ADD rd, x0, rs2	C	1000	rd # 0	rs2 # 0	10
	C.ADD	C.ADD rd, rs2	ADD rd, rd, rs2	C	1001	rs1/rd # 0	rs2 # 0	10
CI-Type					funct3	imme	rd/rs1	imme
	C.LWSP	C.LWSP rd, uimm	LW rd, uimm(x2)	C	010	uimm[5]	rd # 0	uimm[4:2 7:6]
	C.FLWSP	C.FLWSP frd, uimm	FLW frd, uimm(x2)	C	011	uimm[5]	frd	uimm[4:2 7:6]
	C.LI	C.LI rd, imm	ADDI rd, x0, imm	C	010	imme[5]	rd # 0	imme[4:0]
	C.LUI	C.LUI rd, imm	LUI rd, imm	C	011	nzimme[17]	rd # {0, 2}	nzimme[16:12]
	C.ADDI	C.ADDI rd, imm	ADDI rd, rd, imm	C	000	nzimme[5]	rs1/rd # 0	nzimme[4:0]
	C.ADDI16SP	C.ADDI16SP nzimm	ADDI x2,x2, imm	C	011	nzimme[9]	00010	nzimme[4 6 8:7 5]
	C.SLLI	C.SLLI rd, nzimm	SLLI rd, rd, imm	C	000	nzuimme[5]	rs1/rd # 0	nzuimme[4:0]
CSS-Type					funct3	imme	rs2	opcode
	C.SWSP	C.SWSP rs2, uimm	SW rs2, uimm(x2)	C	110	uimm[5:2 7:6]	rs2	10
	C.FSWSP	C.FSWSP frs2, uimm	FSW frs2, uimm(x2)	C	111	uimm[5:2 7:6]	frs2	10
CIW-Type					funct3	c	rd'	opcode
	C.ADDI4SPN	C.ADDI4SPN rd', imm	ADDI rd', x2, imm	C	000	uzuimme[5:4 9:6 2 3]	rd'	00
CL-Type					funct3	imme	rs1'	imme
	C.LW	C.LW rd', imm(rs1')	LW rd', imm(rs1')	C	010	uimm[5:3]	rs1'	uimm[2 6]
	C.FLW	C.FLW frd', uimm(rs1')	FLW frd', uimm(rs1')	C	011	uimm[5:3]	rs1'	uimm[2 6]
CS-Type					funct3	imme	rs1'/rd'	rs2'
	C.SW	C.SW rs2', imm(rs1')	SW rs2', imm(rs1')	C	110	uimm[5:3]	rs1'	uimm[2 6]
	C.FSW	C.FSW frs2', uimm(rs1')	FSW frs2', uimm(rs1')	C	111	uimm[5:3]	rs1'	uimm[2 6]
CA-Type					funct6	rs1'/rd'	funct2	rs2'
	C.AND	C.AND rd', rs2'	AND rd', rd', rs2'	C	100011	rs1'/rd'	11	rs2'
	C.OR	C.OR rd', rs2'	OR rd', rd', rs2'	C	100011	rs1'/rd'	10	rs2'
	C.XOR	C.XOR rd', rs2'	XOR rd', rd', rs2'	C	100011	rs1'/rd'	01	rs2'
	C.SUB	C.SUB rd', rs2'	SUB rd', rd', rs2'	C	100011	rs1'/rd'	00	rs2'
CB-Type					funct3	offset	rs1'/rd'	offset
	C.BEQZ	C.BEQZ rs1', offset	BEQ rs1', x0, offset	C	110	imme[8 4:3]	rs1'	imme[7:6 2:1 5]
	C.BNEZ	C.BNEZ rs1', offset	BNE rs1', x0, offset	C	111	imme[8 4:3]	rs1'	imme[7:6 2:1 5]
					funct3	shamt[5]	funct2	rd'/rs1'
	C.SRLI	C.SRLI rd', imm	SRLI rd', rd', imm	C	100	0	00	rs1'/rd'
	C.SRAI	C.SRAI rd', imm	SRAI rd', rd', imm	C	100	0	01	rs1'/rd'
					funct3	imme[5]	funct2	rd'/rs1'
CJ-Type					funct3	imme[5]	10	rs1'/rd'
	C.J	C.J imm	JAL x0, imm	C	101	imme[11 4 9:8 10 6 7 3:1 5]		01
	C.JAL	C.JAL offset	JAL x1, offset	C	001	imme[11 4 9:8 10 6 7 3:1 5]		01

Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

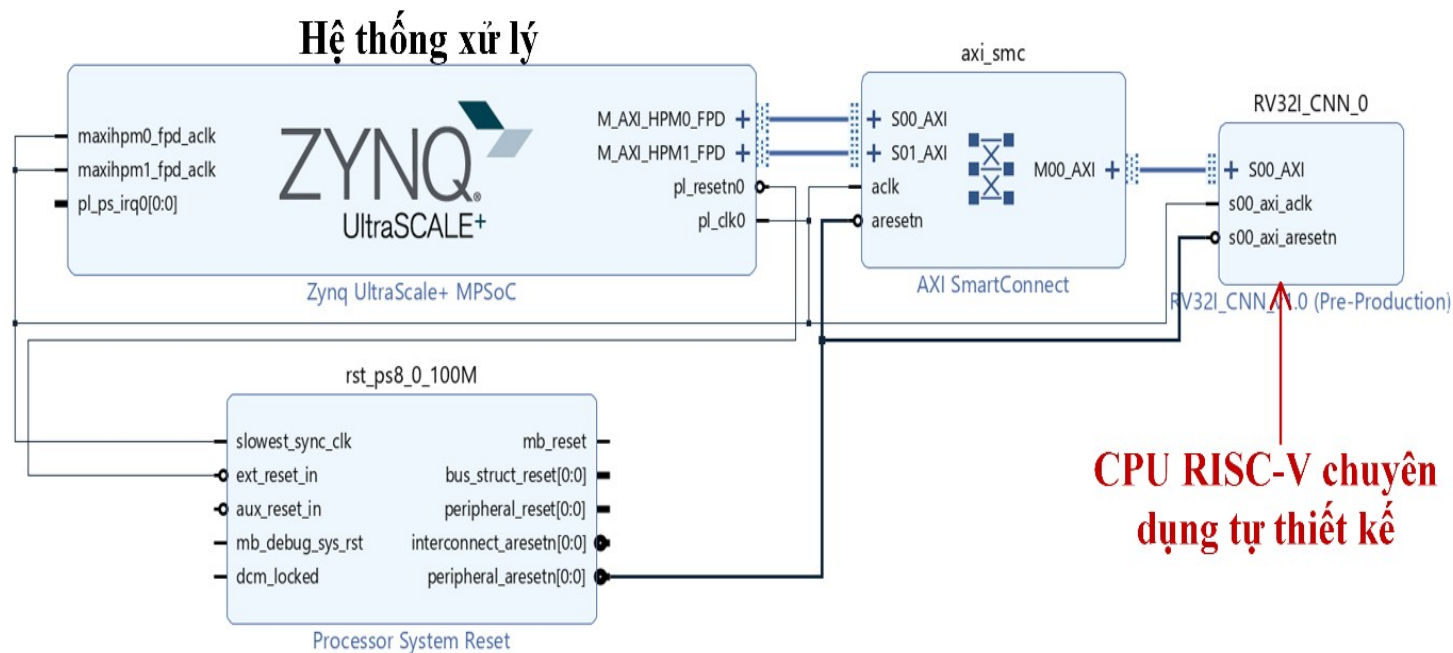
- RV32I và RV64I (Base Integer Instruction Sets):
- RV32E và RV64E: Là một phiên bản thu gọn của RV32I, RV64E: Tương tự như RV32E nhưng với bộ lệnh 64-bit
- RV32M và RV64M (M-Extensions): Các mở rộng M cho phép vi xử lý hỗ trợ các phép toán số học dấu phẩy động (multiplication và division) cho các bộ xử lý 32-bit (RV32M) và 64-bit (RV64M).
- RV32F và RV64F (F-Extensions): Các mở rộng F hỗ trợ các phép toán dấu phẩy động đơn (single-precision floating point).
- RV32D và RV64D (D-Extensions): D-Extensions hỗ trợ các phép toán dấu phẩy động kép (double-precision floating point).
- RV32A và RV64A (A-Extensions): A-Extensions cung cấp hỗ trợ cho các phép toán liên quan đến quản lý bộ nhớ như các lệnh cho phép xác định và thao tác với các địa chỉ bộ nhớ theo cách tối ưu.
- RV32V và RV64V (V-Extensions): V-Extensions mở rộng khả năng hỗ trợ xử lý vectơ cho các ứng dụng cần tính toán song song, chẳng hạn như các ứng dụng AI hoặc máy học.
- RV32C và RV64C (C-Extensions): C-Extensions là các mở rộng giúp giảm kích thước mã lệnh
- RV32G và RV64G (General Extensions): G-Extensions là sự kết hợp của tất cả các mở rộng như M, A, F, D, và C



Tích hợp vi xử lý tự thiết kế vào hệ thống SoC

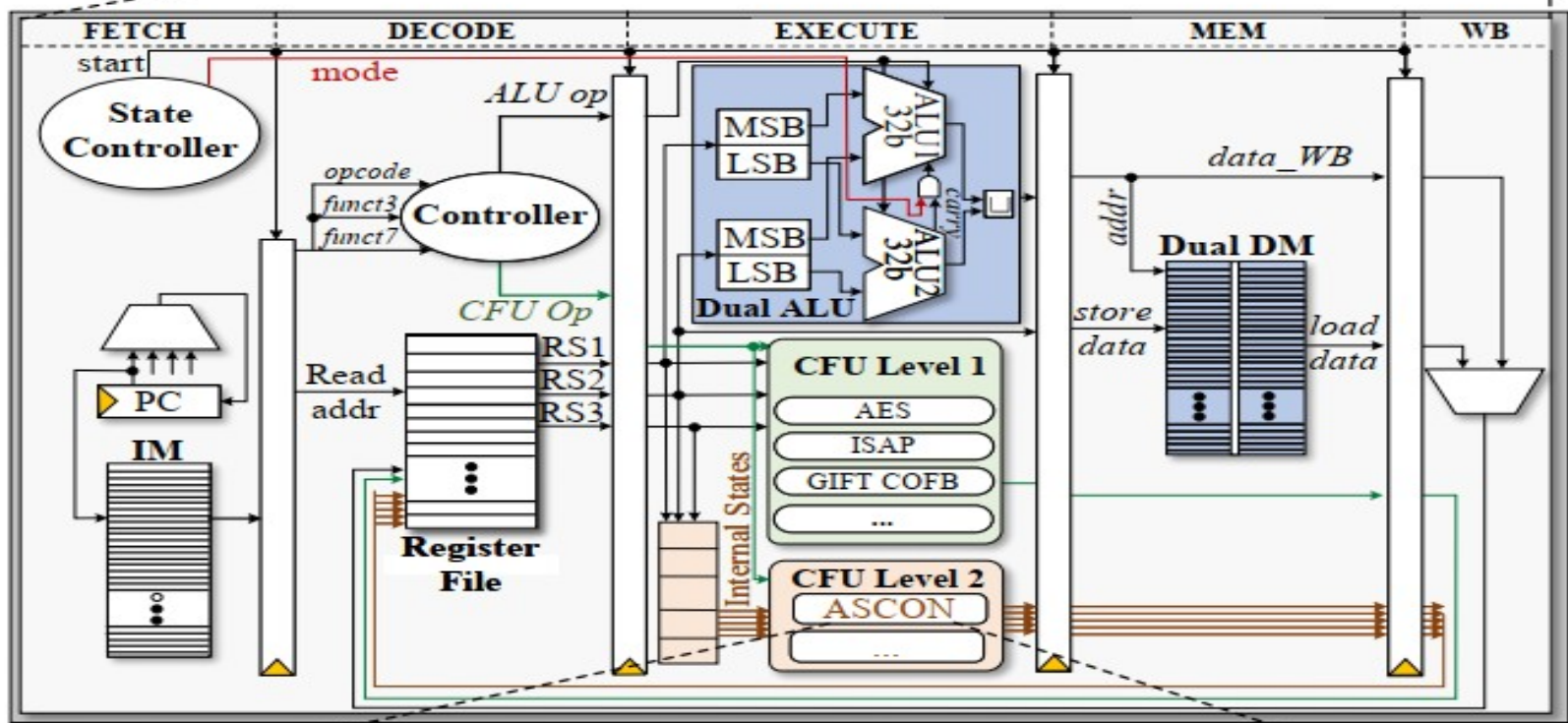
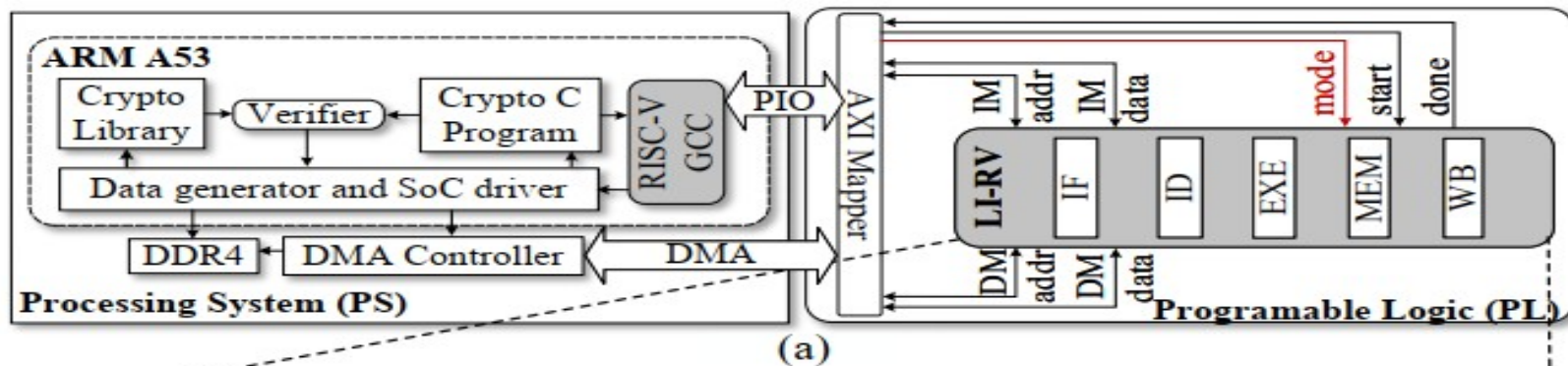


Tích hợp vi xử lý tự thiết kế vào hệ thống SoC



**CPU RISC-V chuyên
dụng tự thiết kế**

Đây có phải là một hệ thống SoC? Nếu phải thì Nhìn hình cho biết hệ thống SoC sau gồm những IP gì. Trong hình này có những dạng vi xử lý gì?



Giới thiệu

Bảng 2.1: So sánh 3 loại vi xử lý trên SoC FPGA

<i>Tiêu chí</i>	<i>Vi xử lý đa năng lõi cứng</i>	<i>Vi xử lý đa năng lõi mềm</i>	<i>Vi xử lý chuyên dụng (DPU)</i>
Tần số hoạt động	Thường cao, có thể lên đến 1.3 GHz hoặc hơn.	Thường thấp, khoảng 100-200 MHz.	Thường trung bình, khoảng 300 MHz.
Tiêu tốn tài nguyên phần cứng FPGA	Không chiếm dụng tài nguyên FPGA.	Tiêu tốn vừa phải tài nguyên FPGA.	Tiêu tốn nhiều tài nguyên FPGA
Khả năng chạy hệ điều hành Linux	Có thể chạy Linux.	Có thể chạy Linux.	Thường không chạy Linux.
Công dụng	Thực hiện các tác vụ tính toán phức tạp, hỗ trợ đa nhiệm.	Tùy chỉnh cho các ứng dụng cụ thể, linh hoạt.	Tăng tốc các tác vụ AI/ML, xử lý dữ liệu.
Khả năng tùy chỉnh phần cứng	Không thể tùy chỉnh.	Cao, có thể tùy chỉnh theo yêu cầu.	Hạn chế.
Tiêu thụ năng lượng	Trung bình	Trung bình đến cao	Cao